

Comprehensive Guide to C Programming

This guide provides a beginner-friendly, structured, and practical exploration of the C programming language, covering all syllabus topics. Each section includes a basic explanation of the concept, comparisons between similar concepts to highlight their differences and use cases, practical examples with expected outputs, and alternative approaches. The goal is to ensure a thorough understanding of C programming through clear explanations, practical examples, and visible results.

Module 1: Introduction to C Programming

1.1 Problem Solving with C in the Era of AI

Explanation:

C remains relevant even with modern technologies like artificial intelligence (AI). AI often requires high-performance programs, and C excels in speed because it compiles directly to machine code, offering low-level hardware control. It's widely used in embedded systems (e.g., IoT devices) and helps developers understand core computing concepts, enhancing problem-solving skills for complex challenges.

Applications:

- **Speed in AI:** Libraries like TensorFlow use C for performance-critical operations.
- **Embedded Systems:** C is ideal for devices like sensors due to its minimal memory footprint.
- **Learning:** Mastering C provides insights into memory management and hardware interaction.

1.2 Salient Features of C

Explanation:

C is favored for its unique characteristics:

- **Procedural:** Code is organized into functions, promoting a clear, step-by-step approach.
- **Fast:** Direct compilation to machine code ensures high performance.
- **Portable:** C code can run on different systems with minimal changes.
- **Low-Level Access:** Allows direct memory manipulation, offering control but requiring caution.
- **Modular:** Functions enable code reuse and organization.

1.3 Program Development Life Cycle (PDLC)

Explanation:

The Program Development Life Cycle (PDLC) outlines the steps to create, test, and maintain a C program:

1. **Requirement Analysis:** Define the problem (e.g., "Calculate the sum of numbers").
 2. **Design:** Plan the solution, such as choosing variables and functions.
 3. **Coding:** Write the C program in a .c file.
 4. **Compilation:** Use a compiler (e.g., GCC) to translate the code into machine language, checking for syntax errors.
 5. **Linking:** Combine the compiled code with necessary libraries (e.g., stdio.h for printf).
 6. **Testing:** Run the program to verify it works as expected, debugging any issues.
 7. **Maintenance:** Update the program as needed (e.g., fixing bugs or adding features).
- PDLC vs Agile Development:** PDLC is a linear, structured approach, ideal for small, well-defined projects. Agile is iterative, better for larger projects with evolving requirements.

Example: A simple program to sum two numbers.

- **Requirement:** Sum two user-input numbers.
- **Design:** Use int variables, scanf for input, and printf for output.
- **Coding:**

```
#include <stdio.h>
int main() {
    int a, b;
    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);
    printf("Sum: %d\n", a + b);
    return 0;
}
// Input: 5 3
// Expected Output: Enter two numbers: Sum: 8
```

- **Compilation:** gcc sum.c -o sum.
- **Linking:** Links with stdio.h functions.
- **Testing:** Run ./sum, input 5 3, check output.
- **Maintenance:** Add error handling if needed.

1.4 Program Structure, Characteristics, and Errors

Explanation:

A C program follows a standard structure:

- #include directives add libraries (e.g., stdio.h).
- The main function is the entry point.
- Code inside main executes, ending with return 0 to indicate success.
Errors in C include:
 - **Syntax Errors:** Code mistakes (e.g., missing ;).
 - **Runtime Errors:** Issues during execution (e.g., division by zero).
 - **Linker Errors:** Missing library functions.
 - **Logical Errors:** Incorrect logic producing wrong results.

Example:

```
#include <stdio.h>
int main() {
    int x = 5;
    printf("Value of x: %d\n", x);
    return 0;
}
// Expected Output: Value of x: 5
```

- **Syntax Error:** printf("Hello") (missing ;).
- **Runtime Error:** int x = 0; printf("%d", 5/x); (division by zero).
- **Logical Error:** int avg = 5 / 2; printf("%d\n", avg);
 - Expected Output: 2 (should be 2.5; fix with 5.0 / 2).

1.5 Variables, Data Types, and Qualifiers

Explanation:

Variables store data, and their type must be declared:

- int: Whole numbers (e.g., 5).
- char: Single characters (e.g., 'A').
- float/double: Decimal numbers (e.g., 3.14).
Qualifiers modify types:
 - unsigned: Restricts to non-negative values, extending the positive range.
 - long: Increases storage capacity for larger numbers.

Data Types and Ranges:

Type	Size	Format Specifier	Range
int	4 bytes	%d	-2,147,483,648 to 2,147,483,647
unsigned int	4 bytes	%u	0 to 4,294,967,295
char	1 byte	%c	-128 to 127
float	4 bytes	%f	~6-7 decimal digits
double	8 bytes	%lf	~15-16 decimal digits

Example:

```
#include <stdio.h>
int main() {
    int age = 25;
    unsigned int count = 100;
    float price = 19.99;
    printf("Age: %d, Count: %u, Price: %.2f\n", age, count, price);
    return 0;
}
```

```
}  
// Expected Output: Age: 25, Count: 100, Price: 19.99
```

1.6 Input/Output Functions

Explanation:

Input/output functions enable user interaction:

- **printf:** Outputs data with format specifiers (e.g., %d for int).
- **scanf:** Reads user input, requiring & to specify variable addresses.
- **getchar/putchar:** Handle single characters.
scanf vs getchar: Use scanf for formatted input (e.g., numbers, strings), while getchar is better for character-by-character processing (e.g., parsing input streams).

Examples:

- **Output:**

```
#include <stdio.h>  
int main() {  
    int num = 42;  
    printf("The number is %d\n", num);  
    return 0;  
}  
// Expected Output: The number is 42
```

- **Input with scanf:**

```
#include <stdio.h>  
int main() {  
    int age;  
    printf("Enter your age: ");  
    scanf("%d", &age);  
    printf("You are %d years old\n", age);  
    return 0;  
}  
// Input: 30  
// Expected Output: Enter your age: You are 30 years old
```

- **Character I/O:**

```
#include <stdio.h>  
int main() {  
    char ch = getchar();  
    putchar(ch);  
    putchar('\n');  
    return 0;  
}  
// Input: A  
// Expected Output: A
```

1.7 Operators and Expressions

Explanation:

Operators perform operations:

- **Arithmetic:** +, -, *, /, %.
- **Relational:** ==, >, <.
- **Logical:** &&, ||.
- **Bitwise:** &, |.
- **Ternary:** condition ? value1 : value2.

An expression combines operators and variables to produce a value.

Ternary vs if-else: The ternary operator is concise for simple assignments, while if-else suits complex logic with multiple statements.

Examples:

- **Arithmetic:**

```
#include <stdio.h>
int main() {
    int sum = 5 + 3;
    printf("Sum: %d\n", sum);
    return 0;
}
// Expected Output: Sum: 8
```

- **Relational:**

```
#include <stdio.h>
int main() {
    int x = 10, y = 5;
    if (x > y) printf("x is greater\n");
    return 0;
}
// Expected Output: x is greater
```

- **Ternary vs if-else:**

```
#include <stdio.h>
int main() {
    int x = 10, y = 5;
    int max = (x > y) ? x : y; // Ternary: concise
    printf("Max (ternary): %d\n", max);
    if (x > y) { // if-else: more statements
        printf("x is greater\n");
        max = x;
    } else {
        printf("y is greater or equal\n");
        max = y;
    }
    printf("Max (if-else): %d\n", max);
    return 0;
}
// Expected Output:
// Max (ternary): 10
// x is greater
// Max (if-else): 10
```

1.8 Language Specifications and Coding Standards

Explanation:

C has rules to prevent errors:

- **Undefined Behavior:** Actions like accessing out-of-bounds array elements cause unpredictable results.
- **Coding Standards:** Enhance readability:
 - Use snake_case for identifiers (e.g., my_variable).
 - Indent consistently (e.g., 4 spaces).
 - Add comments for clarity.

Examples:

- **Undefined Behavior:**

```
#include <stdio.h>
```

```
int main() {
    int arr[3];
    arr[5] = 10; // Out of bounds
    printf("%d\n", arr[5]); // Undefined
    return 0;
}
// Expected Output: (Unpredictable, may crash)
```

- **Coding Standard:**

```
#include <stdio.h>
int main() {
    // Clear variable name and indentation
    int total_score = 0;
    for (int i = 0; i < 5; i++) {
        total_score += i;
    }
    printf("Total: %d\n", total_score);
    return 0;
}
// Expected Output: Total: 10
```

Module 2: Control Structures

2.1 Selection Statements

Explanation:

Selection statements enable decision-making:

- **if:** Executes code if a condition is true.
- **if-else:** Provides an alternative if the condition is false.
- **switch:** Selects from multiple options based on a value.
switch vs if-else: Use switch for a single variable with discrete values (e.g., menu options) for readability and potential compiler optimizations. Use if-else for complex conditions involving ranges or multiple variables.

Examples:

- **if-else:**

```
#include <stdio.h>
int main() {
    int temp = 35;
    if (temp > 30) {
        printf("It's hot!\n");
    } else {
        printf("It's cool!\n");
    }
    return 0;
}
// Expected Output: It's hot!
```

- **switch:**

```
#include <stdio.h>
int main() {
    int choice = 2;
    switch (choice) {
        case 1: printf("Option 1\n"); break;
        case 2: printf("Option 2\n"); break;
        default: printf("Invalid\n"); break;
    }
    return 0;
}
// Expected Output: Option 2
```

- **Comparison:**

```
#include <stdio.h>
int main() {
    int choice = 2;
    // switch: cleaner for discrete values
    switch (choice) {
        case 1: printf("Option 1\n"); break;
        case 2: printf("Option 2\n"); break;
        default: printf("Invalid\n"); break;
    }
    // if-else: more flexible
    if (choice == 1) printf("Option 1\n");
    else if (choice == 2) printf("Option 2\n");
    else printf("Invalid\n");
    return 0;
}
// Expected Output:
// Option 2
// Option 2
```

2.2 Looping Statements

Explanation:

Loops repeat code based on conditions:

- **for:** Ideal for a known number of iterations.
- **while:** Repeats while a condition is true.
- **do-while:** Ensures at least one iteration.
for vs while vs do-while: Use for for fixed iterations (e.g., array traversal). Use while for condition-driven loops (e.g., reading until a sentinel). Use do-while when the loop must run at least once (e.g., user prompts).

Examples:

- **for:**

```
#include <stdio.h>
int main() {
    for (int i = 1; i <= 3; i++) {
        printf("%d ", i);
    }
    printf("\n");
    return 0;
}
// Expected Output: 1 2 3
```

- **while:**

```
#include <stdio.h>
int main() {
    int i = 1;
    while (i <= 3) {
        printf("%d ", i++);
    }
    printf("\n");
    return 0;
}
// Expected Output: 1 2 3
```

- **do-while:**

```
#include <stdio.h>
```

```
int main() {
    int choice = 1;
    do {
        printf("Loop iteration\n");
        choice = 0;
    } while (choice != 0);
    return 0;
}
// Expected Output: Loop iteration
```

- **Comparison:**

```
#include <stdio.h>
int main() {
    // for: fixed iterations
    for (int i = 1; i <= 3; i++) printf("for: %d\n", i);
    // while: condition-based
    int j = 1;
    while (j <= 3) printf("while: %d\n", j++);
    // do-while: at least one run
    int k = 0;
    do printf("do-while: %d\n", k); while (k > 0);
    return 0;
}
// Expected Output:
// for: 1
// for: 2
// for: 3
// while: 1
// while: 2
// while: 3
// do-while: 0
```

2.3 Problem Solving Using Control Structures

Explanation:

Control structures solve problems by enabling decisions and repetition. For example, determine if a number is even or odd.

Example:

```
#include <stdio.h>
int main() {
    int num = 4;
    if (num % 2 == 0) {
        printf("%d is even\n", num);
    } else {
        printf("%d is odd\n", num);
    }
    return 0;
}
// Expected Output: 4 is even
```

Single Character I/O Problem: Echo characters until 'q'.

```
#include <stdio.h>
int main() {
    char ch;
    do {
        ch = getchar();
        putchar(ch);
    } while (ch != 'q');
    return 0;
}
// Input: abq
// Expected Output: abq
```

Module 3: Arrays, Pointers, and Functions

3.1 Arrays: 1D, Initialization, and Traversal

Explanation:

A 1D array is a list of elements of the same type. You can initialize it at declaration and traverse it using loops.

Example:

```
#include <stdio.h>
int main() {
    int arr[5] = {10, 20, 30, 40, 50};
    for (int i = 0; i < 5; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
    return 0;
}
// Expected Output: 10 20 30 40 50
```

3.2 Arrays: 2D

Explanation:

A 2D array represents a grid (rows and columns), useful for matrices or tables.

1D vs 2D Arrays: Use 1D arrays for linear data (e.g., a list of scores). Use 2D arrays for grid-like data (e.g., a chessboard).

Example:

```
#include <stdio.h>
int main() {
    int matrix[2][3] = {{1, 2, 3}, {4, 5, 6}};
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 3; j++) {
            printf("%d ", matrix[i][j]);
        }
        printf("\n");
    }
    return 0;
}
// Expected Output:
// 1 2 3
// 4 5 6
```

• Comparison:

```
#include <stdio.h>
int main() {
    // 1D: simple list
    int arr[3] = {1, 2, 3};
    for (int i = 0; i < 3; i++) printf("%d ", arr[i]);
    printf("\n");
    // 2D: grid
    int matrix[2][2] = {{1, 2}, {3, 4}};
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) printf("%d ", matrix[i][j]);
        printf("\n");
    }
    return 0;
}
// Expected Output:
// 1 2 3
// 1 2
// 3 4
```


3.3 Problem Solving Using Arrays

Explanation:

Arrays are useful for list-based problems, like calculating averages.

Example:

```
#include <stdio.h>
int main() {
    int arr[4] = {10, 20, 30, 40};
    int sum = 0;
    for (int i = 0; i < 4; i++) {
        sum += arr[i];
    }
    printf("Average: %.2f\n", (float)sum / 4);
    return 0;
}
// Expected Output: Average: 25.00
```

3.4 Pointers

Explanation:

Pointers store memory addresses. Use * to access the value at the address and & to get a variable's address.

Pointers vs Arrays: Arrays provide fixed, contiguous memory with index-based access, while pointers offer dynamic memory manipulation (e.g., for dynamic allocation).

Example:

```
#include <stdio.h>
int main() {
    int x = 10;
    int *p = &x;
    printf("Value: %d\n", *p);
    *p = 20;
    printf("New value of x: %d\n", x);
    return 0;
}
// Expected Output:
// Value: 10
// New value of x: 20
```

- **Comparison:**

```
#include <stdio.h>
int main() {
    // Array: fixed size
    int arr[3] = {10, 20, 30};
    printf("Array: %d\n", arr[1]);
    // Pointer: dynamic
    int x = 20;
    int *p = &x;
    printf("Pointer: %d\n", *p);
    return 0;
}
// Expected Output:
// Array: 20
// Pointer: 20
```

3.5 Array of Pointers

Explanation:

An array of pointers stores pointers, often used for lists of strings.

Array of Pointers vs 2D Array for Strings: An array of pointers is memory-efficient for strings of varying lengths, while a 2D array allocates fixed space per string, potentially wasting memory.

Example:

```
#include <stdio.h>
int main() {
    const char *names[] = {"Alice", "Bob", "Charlie", NULL};
    for (int i = 0; names[i]; i++) {
        printf("%s\n", names[i]);
    }
    return 0;
}
// Expected Output:
// Alice
// Bob
// Charlie
```

- **Comparison:**

```
#include <stdio.h>
int main() {
    // Array of pointers: variable length
    const char *names1[] = {"Al", "Bobby", "Charlie", NULL};
    for (int i = 0; names1[i]; i++) printf("%s ", names1[i]);
    printf("\n");
    // 2D array: fixed length
    char names2[3][10] = {"Al", "Bobby", "Charlie"};
    for (int i = 0; i < 3; i++) printf("%s ", names2[i]);
    printf("\n");
    return 0;
}
// Expected Output:
// Al Bobby Charlie
// Al Bobby Charlie
```

3.6 Functions

Explanation:

Functions encapsulate reusable code for specific tasks, improving modularity.

Example:

```
#include <stdio.h>
int add(int a, int b) {
    return a + b;
}
int main() {
    int sum = add(5, 3);
    printf("Sum: %d\n", sum);
    return 0;
}
// Expected Output: Sum: 8
```

3.7 Arrays and Functions

Explanation:

Passing an array to a function passes a pointer to its first element, allowing the function to modify the array.

Example:

```
#include <stdio.h>
void double_array(int arr[], int size) {
    for (int i = 0; i < size; i++) {
```

```

        arr[i] *= 2;
    }
}
int main() {
    int arr[] = {1, 2, 3};
    double_array(arr, 3);
    for (int i = 0; i < 3; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
    return 0;
}
// Expected Output: 2 4 6

```

3.8 Problem Solving Using Arrays, Functions, and Pointers

Explanation:

Combining arrays, functions, and pointers enables efficient solutions, like reversing an array.

Example:

```

#include <stdio.h>
void reverse(int *arr, int size) {
    int *start = arr, *end = arr + size - 1;
    while (start < end) {
        int temp = *start;
        *start++ = *end;
        *end-- = temp;
    }
}
int main() {
    int arr[] = {1, 2, 3, 4};
    reverse(arr, 4);
    for (int i = 0; i < 4; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
    return 0;
}
// Expected Output: 4 3 2 1

```

3.9 Storage Classes

Explanation:

Storage classes define variable scope and lifetime:

- **auto:** Default for local variables, destroyed after function exit.
 - **static:** Retains value between calls.
 - **extern:** Shares variables across files.
 - **register:** Suggests CPU register storage for speed.
- static vs auto:** Use static for persistent values (e.g., counters). Use auto for temporary variables.

Example (static):

```

#include <stdio.h>
void count_calls() {
    static int count = 0;
    count++;
    printf("Called %d times\n", count);
}
int main() {
    count_calls();
    count_calls();
    return 0;
}
// Expected Output:
// Called 1 times
// Called 2 times

```

- **Comparison:**

```
#include <stdio.h>
void auto_counter() {
    int count = 0;
    count++;
    printf("auto: %d\n", count);
}
void static_counter() {
    static int count = 0;
    count++;
    printf("static: %d\n", count);
}
int main() {
    auto_counter();
    auto_counter();
    static_counter();
    static_counter();
    return 0;
}
// Expected Output:
// auto: 1
// auto: 1
// static: 1
// static: 2
```

3.10 Recursion

Explanation:

Recursion involves a function calling itself, requiring a base case to terminate.

Recursion vs Iteration: Use recursion for naturally recursive problems (e.g., factorials). Use iteration for better performance in linear tasks (e.g., summing numbers).

Example: Factorial.

```
#include <stdio.h>
int factorial(int n) {
    if (n <= 1) return 1;
    return n * factorial(n - 1);
}
int main() {
    printf("Factorial of 4: %d\n", factorial(4));
    return 0;
}
// Expected Output: Factorial of 4: 24
```

- **Comparison:**

```
#include <stdio.h>
int factorial_recursive(int n) {
    if (n <= 1) return 1;
    return n * factorial_recursive(n - 1);
}
int factorial_iterative(int n) {
    int result = 1;
    for (int i = 1; i <= n; i++) result *= i;
    return result;
}
int main() {
    printf("Recursive: %d\n", factorial_recursive(4));
    printf("Iterative: %d\n", factorial_iterative(4));
    return 0;
}
// Expected Output:
// Recursive: 24
// Iterative: 24
```

3.11 Callbacks Using Function Pointers

Explanation:

Function pointers point to functions, enabling callbacks for flexible behavior.

Function Pointers vs Direct Calls: Use function pointers for dynamic behavior (e.g., custom sorting). Use direct calls for fixed, simple functions.

Example:

```
#include <stdio.h>
typedef int (*Operation)(int);
int square(int x) { return x * x; }
void apply(int x, Operation op) {
    printf("Result: %d\n", op(x));
}
int main() {
    apply(4, square);
    return 0;
}
// Expected Output: Result: 16
```

- **Comparison:**

```
#include <stdio.h>
int square(int x) { return x * x; }
void apply_direct(int x) {
    printf("Direct: %d\n", square(x));
}
typedef int (*Operation)(int);
void apply_callback(int x, Operation op) {
    printf("Callback: %d\n", op(x));
}
int main() {
    apply_direct(4);
    apply_callback(4, square);
    return 0;
}
// Expected Output:
// Direct: 16
// Callback: 16
```

Module 4: Searching and Sorting

4.1 Searching

Explanation:

Searching locates an item in a list:

- **Linear Search:** Checks each element sequentially.
- **Binary Search:** Requires a sorted list, dividing the search space in half each step.
Linear Search vs Binary Search: Use linear search for unsorted or small lists (simpler). Use binary search for large, sorted lists (faster, $O(\log n)$).

Examples:

- **Linear Search:**

```
#include <stdio.h>
int linear_search(int arr[], int size, int target) {
    for (int i = 0; i < size; i++) {
        if (arr[i] == target) return i;
    }
    return -1;
}
int main() {
    int arr[] = {5, 2, 8, 1};
```

```

    int pos = linear_search(arr, 4, 8);
    printf("Position: %d\n", pos);
    return 0;
}
// Expected Output: Position: 2

```

• Binary Search:

```

#include <stdio.h>
int binary_search(int arr[], int size, int target) {
    int low = 0, high = size - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] == target) return mid;
        if (arr[mid] < target) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}
int main() {
    int arr[] = {1, 3, 5, 7};
    int pos = binary_search(arr, 4, 5);
    printf("Position: %d\n", pos);
    return 0;
}
// Expected Output: Position: 2

```

• Comparison:

```

#include <stdio.h>
int linear_search(int arr[], int size, int target) {
    for (int i = 0; i < size; i++) if (arr[i] == target) return i;
    return -1;
}
int binary_search(int arr[], int size, int target) {
    int low = 0, high = size - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] == target) return mid;
        if (arr[mid] < target) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}
int main() {
    int unsorted[] = {5, 2, 8, 1};
    int sorted[] = {1, 3, 5, 7};
    printf("Linear (unsorted): %d\n", linear_search(unsorted, 4, 8));
    printf("Binary (sorted): %d\n", binary_search(sorted, 4, 5));
    return 0;
}
// Expected Output:
// Linear (unsorted): 2
// Binary (sorted): 2

```

4.2 Sorting

Explanation:

Sorting orders a list. C's qsort function sorts efficiently, using a comparison callback.

Example:

```

#include <stdio.h>
#include <stdlib.h>
int compare(const void *a, const void *b) {
    return (*(int*)a - *(int*)b);
}

```

```
int main() {
    int arr[] = {4, 1, 3, 2};
    qsort(arr, 4, sizeof(int), compare);
    for (int i = 0; i < 4; i++) printf("%d ", arr[i]);
    printf("\n");
    return 0;
}
// Expected Output: 1 2 3 4
```

Sorting Strings:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
int compare_strings(const void *a, const void *b) {
    return strcmp(*(const char**)a, *(const char**)b);
}
int main() {
    const char *names[] = {"Bob", "Alice", "Charlie"};
    qsort(names, 3, sizeof(const char*), compare_strings);
    for (int i = 0; i < 3; i++) printf("%s ", names[i]);
    printf("\n");
    return 0;
}
// Expected Output: Alice Bob Charlie
```

Module 5: Strings and Command Line Arguments

5.1 Strings in C

Explanation:

A string in C is a character array ending with `\0`. Strings can be initialized as arrays, pointers, or dynamically.

Array vs Pointer for Strings: Use arrays for modifiable strings. Use pointers for read-only string literals to save memory.

Examples:

- **Array:**

```
#include <stdio.h>
int main() {
    char str[] = "Hello";
    str[0] = 'h';
    printf("%s\n", str);
    return 0;
}
// Expected Output: hello
```

- **Pointer:**

```
#include <stdio.h>
int main() {
    const char *str = "Hello";
    printf("%s\n", str);
    return 0;
}
// Expected Output: Hello
```

5.2 String Manipulation Functions

Explanation:

The `<string.h>` library provides string functions:

- `strlen`, `strcmp`, `strncmp`, `strcpy`, `strncpy`, `strcat`, `strncat`, `strchr`, `strrchr`, `strstr`, `strtok`, `strspn`, `strcspn`, `strpbrk`.
strcpy VS strncpy: Use `strcpy` for simple copying, but `strncpy` to limit characters copied, preventing overflows (ensure null termination).

Examples:

- `strlen`:

```
#include <stdio.h>
#include <string.h>
int main() {
    char str[] = "Hello";
    printf("Length: %zu\n", strlen(str));
    return 0;
}
// Expected Output: Length: 5
```

- `strcpy` VS `strncpy`:

```
#include <stdio.h>
#include <string.h>
int main() {
    char dest1[10], dest2[10] = {0};
    strcpy(dest1, "Hello"); // Simple copy
    strncpy(dest2, "Hello", 5); // Safer, limited copy
    printf("strcpy: %s\n", dest1);
    printf("strncpy: %s\n", dest2);
    return 0;
}
// Expected Output:
// strcpy: Hello
// strncpy: Hello
```

5.3 Errors and Best Practices

Explanation:

String errors include:

- **Buffer Overflow:** Writing beyond allocated space.
- **Missing `\0`:** Causes undefined behavior.
Best Practices: Use `fgets` for safe input, `sprintf` to avoid overflows.
`fgets` VS `scanf`: `fgets` prevents overflow by limiting input, while `scanf` with `%s` risks overflow.

Example:

```
#include <stdio.h>
#include <string.h>
int main() {
    char name[50];
    printf("Enter name: ");
    fgets(name, 50, stdin);
    name[strcspn(name, "\n")] = '\0';
    printf("Hello, %s\n", name);
    return 0;
}
// Input: Alice
// Expected Output:
// Enter name: Hello, Alice
```

5.4 Command Line Arguments

Explanation:

Command line arguments are passed to `main` via `argc` (argument count) and `argv` (argument vector).

Example:

```
#include <stdio.h>
int main(int argc, char *argv[]) {
    if (argc != 2) {
        printf("Usage: %s <name>\n", argv[0]);
        return 1;
    }
    printf("Hello, %s!\n", argv[1]);
    return 0;
}
// Run: ./program Alice
// Expected Output: Hello, Alice!
```

Module 6: Dynamic Memory Management

6.1 Dynamic Memory Management

Explanation:

Dynamic memory allocation allows runtime memory requests:

- malloc, calloc, realloc, free.
malloc vs calloc: malloc is faster but uninitialized; calloc initializes to zero.

Examples:

- malloc:

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    int *arr = (int*)malloc(3 * sizeof(int));
    arr[0] = 10; arr[1] = 20; arr[2] = 30;
    for (int i = 0; i < 3; i++) printf("%d ", arr[i]);
    printf("\n");
    free(arr);
    return 0;
}
// Expected Output: 10 20 30
```

- calloc:

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    int *arr = (int*)calloc(3, sizeof(int));
    for (int i = 0; i < 3; i++) printf("%d ", arr[i]);
    printf("\n");
    free(arr);
    return 0;
}
// Expected Output: 0 0 0
```

6.2 Simple Linked List

Explanation:

A linked list connects nodes dynamically, allowing flexible size adjustments.

Linked List vs Array: Linked lists are dynamic and efficient for insertions, while arrays offer faster indexed access.

Example:

```
#include <stdio.h>
```

```
#include <stdlib.h>
typedef struct Node {
    int data;
    struct Node *next;
} Node;
void insert(Node **head, int data) {
    Node *new_node = (Node*)malloc(sizeof(Node));
    new_node->data = data;
    new_node->next = *head;
    *head = new_node;
}
void print(Node *head) {
    while (head) {
        printf("%d -> ", head->data);
        head = head->next;
    }
    printf("NULL\n");
}
int main() {
    Node *head = NULL;
    insert(&head, 3);
    insert(&head, 2);
    print(head);
    return 0;
}
// Expected Output: 2 -> 3 -> NULL
```

6.3 Stack

Explanation:

A stack follows LIFO (last in, first out).

Stack vs Queue: Stacks are LIFO (e.g., undo operations); queues are FIFO (e.g., task queues).

Example:

```
#include <stdio.h>
#define SIZE 5
int stack[SIZE], top = -1;
void push(int val) {
    if (top == SIZE - 1) return;
    stack[++top] = val;
}
int pop() {
    if (top == -1) return -1;
    return stack[top--];
}
int main() {
    push(10);
    push(20);
    printf("Popped: %d\n", pop());
    return 0;
}
// Expected Output: Popped: 20
```

6.4 Queue and Priority Queue

Explanation:

A queue follows FIFO (first in, first out). A priority queue dequeues based on priority.

Queue vs Priority Queue: Use queues for strict FIFO; priority queues for priority-based ordering.

Example:

```
#include <stdio.h>
#define SIZE 5
int queue[SIZE], front = 0, rear = -1;
void enqueue(int val) {
    if (rear == SIZE - 1) return;
    queue[++rear] = val;
}
int dequeue() {
    if (front > rear) return -1;
```

```
        return queue[front++];
    }
    int main() {
        enqueue(10);
        enqueue(20);
        printf("Dequeued: %d\n", dequeue());
        return 0;
    }
    // Expected Output: Dequeued: 10
```

Module 7: User-Defined Types

7.1 Structures and Functions

Explanation:

Structures group related data, and functions can operate on them.

Example:

```
#include <stdio.h>
typedef struct {
    char name[50];
    int id;
} Student;
void print_student(Student s) {
    printf("Name: %s, ID: %d\n", s.name, s.id);
}
int main() {
    Student s = {"Alice", 101};
    print_student(s);
    return 0;
}
// Expected Output: Name: Alice, ID: 101
```

7.2 #pragma Directive

Explanation:

#pragma provides compiler instructions, like pack to control structure alignment.

Example:

```
#include <stdio.h>
#pragma pack(push, 1)
struct Packed {
    int x;
    char c;
};
#pragma pack(pop)
int main() {
    struct Packed p = {1, 'A'};
    printf("Size: %zu\n", sizeof(p));
    return 0;
}
// Expected Output: Size: 5 (or smaller, depending on system)
```

7.3 Array of Structures

Explanation:

An array of structures stores multiple records.

Example:

```
#include <stdio.h>
```

```
typedef struct {
    char name[50];
    int id;
} Student;
int main() {
    Student students[2] = {"Bob", 102}, {"Charlie", 103};
    for (int i = 0; i < 2; i++) {
        printf("Student %d: %s\n", students[i].id, students[i].name);
    }
    return 0;
}
// Expected Output:
// Student 102: Bob
// Student 103: Charlie
```

7.4 Problem Solving Using Structures and Functions

Explanation:

Structures and functions can solve problems like sorting records.

Example:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
typedef struct {
    char name[50];
    int id;
} Student;
int compare_students(const void *a, const void *b) {
    return ((Student*)a)->id - ((Student*)b)->id;
}
int main() {
    Student students[3] = {"Charlie", 103}, {"Alice", 101}, {"Bob", 102};
    qsort(students, 3, sizeof(Student), compare_students);
    for (int i = 0; i < 3; i++) printf("%d: %s\n", students[i].id, students[i].name);
    return 0;
}
// Expected Output:
// 101: Alice
// 102: Bob
// 103: Charlie
```

7.5 Sorting and Searching: Array of Structures/Pointers to Structures

Explanation:

Sort or search structures using arrays or pointers.

Array of Structures vs Pointers: Arrays of structures are contiguous and simpler; pointers are flexible for scattered data.

Example:

```
#include <stdio.h>
#include <string.h>
typedef struct {
    char name[50];
    int id;
} Student;
int main() {
    Student s1 = {"Alice", 101}, s2 = {"Bob", 102};
    Student *ptrs[] = {&s1, &s2, NULL};
    for (int i = 0; ptrs[i]; i++) {
        if (ptrs[i]->id == 101) printf("Found: %s\n", ptrs[i]->name);
    }
    return 0;
}
// Expected Output: Found: Alice
```

7.6 Bit Fields

Explanation:

Bit fields allocate specific bits, saving memory for small values.

Bit Fields vs Regular Fields: Use bit fields for memory efficiency (e.g., flags); regular fields for larger data.

Example:

```
#include <stdio.h>
struct Flags {
    unsigned int on : 1;
    unsigned int error : 1;
};
int main() {
    struct Flags f = {1, 0};
    printf("On: %d\n", f.on);
    return 0;
}
// Expected Output: On: 1
```

7.7 Unions

Explanation:

Unions share memory among members, allowing only one to be active.

Unions vs Structures: Use unions for mutually exclusive data (e.g., a value as int or float); structures for coexisting data.

Example:

```
#include <stdio.h>
union Data {
    int i;
    float f;
};
int main() {
    union Data d;
    d.i = 65;
    printf("As char: %c\n", *(char*)&d.i);
    return 0;
}
// Expected Output: As char: A
```

• Comparison:

```
#include <stdio.h>
struct StudentStruct {
    int id;
    float score;
};
union DataUnion {
    int i;
    float f;
};
int main() {
    struct StudentStruct s = {101, 95.5};
    union DataUnion u;
    u.i = 65;
    printf("Struct size: %zu\n", sizeof(s));
    printf("Union size: %zu\n", sizeof(u));
    printf("Struct: ID=%d, Score=%.1f\n", s.id, s.score);
    printf("Union as char: %c\n", *(char*)&u.i);
    return 0;
}
// Expected Output:
// Struct size: 8
// Union size: 4
// Struct: ID=101, Score=95.5
// Union as char: A
```

7.8 Enumerations

Explanation:

Enums define named constants for readability.

Enums vs #define: Enums group related constants with type safety; #define is for simple replacements.

Example:

```
#include <stdio.h>
enum Day {MON, TUE, WED};
int main() {
    enum Day today = TUE;
    printf("Day: %d\n", today);
    return 0;
}
// Expected Output: Day: 1
```

Module 8: File Handling

8.1 File Handling Functions and Error Handling

Explanation:

File handling uses FILE pointers and functions like fopen, fclose, fprintf, fgets, fwrite, fread. Always handle errors (e.g., fopen returning NULL).

Text vs Binary Files: Text files are human-readable; binary files are compact for structured data.

Example:

```
#include <stdio.h>
int main() {
    FILE *fp = fopen("data.txt", "w");
    if (!fp) return 1;
    fprintf(fp, "Hello\n");
    fclose(fp);

    fp = fopen("data.txt", "r");
    char line[100];
    fgets(line, 100, fp);
    printf("Read: %s", line);
    fclose(fp);
    return 0;
}
// Expected Output: Read: Hello
```

8.2 File I/O Using Redirection

Explanation:

Redirection uses files for input/output without code changes (e.g., ./program < input.txt > output.txt).

Example:

```
#include <stdio.h>
int main() {
    int num;
    scanf("%d", &num);
    printf("Number: %d\n", num);
    return 0;
}
// input.txt: 42
// Expected Output (in output.txt): Number: 42
```

8.3 Searching and Sorting in Files Using Array of Pointers

Explanation:

Read file data into an array of pointers, then search or sort.

Example:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MAX_LINES 100
#define MAX_LEN 256
int main() {
    FILE *fp = fopen("data.txt", "w");
    fprintf(fp, "Hello World\nThis is a test\n");
    fclose(fp);

    fp = fopen("data.txt", "r");
    char *lines[MAX_LINES];
    int count = 0;
    char buffer[MAX_LEN];
    while (fgets(buffer, MAX_LEN, fp) && count < MAX_LINES) {
        buffer[strcspn(buffer, "\n")] = '\0';
        lines[count] = (char*)malloc(strlen(buffer) + 1);
        strcpy(lines[count], buffer);
        count++;
    }
    fclose(fp);
    for (int i = 0; i < count; i++) {
        if (strstr(lines[i], "Hello")) printf("Line %d: %s\n", i + 1, lines[i]);
        free(lines[i]);
    }
    return 0;
}
// Expected Output: Line 1: Hello World
```

Module 9: Build Process and Preprocessor

9.1 Header File Creation and Use

Explanation:

Header files (.h) share declarations across files, promoting modularity.

Header Files vs Inline Code: Header files organize reusable code; inline code suits small programs.

Example:

- mymath.h:

```
#ifndef MYMATH_H
#define MYMATH_H
int add(int a, int b);
#endif
```

- mymath.c:

```
#include "mymath.h"
int add(int a, int b) { return a + b; }
```

- main.c:

```
#include <stdio.h>
#include "mymath.h"
```

```
int main() {
    printf("Sum: %d\n", add(2, 3));
    return 0;
}
// Expected Output: Sum: 5
```

9.2 Environment Variables

Explanation:

Environment variables (accessed via `getenv`) allow system-specific adaptations.

Example:

```
#include <stdio.h>
#include <stdlib.h>
int main() {
    char *user = getenv("USER");
    if (!user) user = getenv("USERNAME");
    printf("Hello, %s!\n", user ? user : "Unknown");
    return 0;
}
// Expected Output: Hello, (your username or "Unknown")!
```

9.3 Preprocessor Directives and Conditional Compilation

Explanation:

Preprocessor directives (`#include`, `#define`, `#ifdef`) modify code before compilation.

#define vs Constants: `#define` is for macros and pre-compilation replacements; `const` offers type safety within the program.

Example:

```
#include <stdio.h>
#define DEBUG
int main() {
    #ifdef DEBUG
        printf("Debug mode\n");
    #endif
    return 0;
}
// Expected Output: Debug mode
```

Conclusion

This guide covers C programming with detailed explanations, examples, outputs, and comparisons. Practice by building projects like a calculator or to-do list to master C!